.NET Web Backend Mastery Plan

A 24-week roadmap from working developer to confident backend engineer

Duration	24 weeks (adjustable)
Time needed	About 10-12 hours per week. If you have less time, stretch each phase by 1.5x instead of skipping topics.
Method	Learn a concept, build it in a real project, break it on purpose, then explain it back in your own words.
Practice ground	Your Mercato e-commerce project (Clean Architecture, CQRS + MediatR, .NET 10). Every phase ends with a concrete deliverable there.
Language	Formal English for content, short Roman Urdu notes for the ideas that matter most.

An honest note on 'master everything'
No one knows every corner of .NET, and that is fine. The realistic goal is a T-shaped engineer: broad and solid across all layers of a backend, and genuinely deep in a few (data access, security and architecture are the best candidates).

Roman Urdu: Har cheez ratt lena maqsad nahi. Maqsad ye hai ke koi bhi backend problem aaye to tum jaano ke kahan dekhna hai, kya measure karna hai, aur sahi solution kaise chunna hai.

Roadmap at a Glance

#	Phase	When	Proof you are done
0	Baseline audit and setup	Week 1	Rate yourself, set up learning log and lab repo
1	C# and .NET Internals	Weeks 2-3	Explain, without notes, why .Result can deadlock and what happens step by step when you await a Task.
2	ASP.NET Core Deep Dive	Weeks 4-6	Draw the request pipeline from memory and explain why UseAuthentication must come before UseAuthorization.
3	Data Access Mastery	Weeks 7-10	Take one slow endpoint, prove the N+1 from logs, fix it, and record query count and response time before and after.
4	Authentication and Security	Weeks 11-13	Attack your own API: try IDOR, token replay and mass assignment. Document each finding and fix it.
5	Architecture and Design Patterns	Weeks 14-16	Write three Architecture Decision Records (ADRs) for your project, each comparing your choice against a real alternative.
6	Performance, Reliability and Real-Time	Weeks 17-19	Load test with k6, find the top bottleneck using traces, fix it, and show the improvement in p95 latency.
7	Testing	Week 20 (and continuous)	A CI gate that fails the build when a test fails or an architecture rule is broken.
8	DevOps and Deployment	Weeks 21-22	A push to main automatically runs tests, builds the image and deploys to staging.
9	Advanced and Distributed Systems	Weeks 23-24	Write a design document describing how Mercato would scale to 10x traffic, including what you would NOT change.

Phases 1 to 3 build the foundation, 4 and 5 make it secure and well designed, 6 to 8 make it production-ready, and 9 stretches you into system design. Testing (Phase 7) should also be practised continuously from Phase 2 onward, not only in Week 20.

Phase 0: Baseline Audit (Week 1)

Before starting, find out where you really stand. Rate every topic in the checklist at the end of this document from 1 (never used) to 5 (could teach it). Then:

- Pick your **three weakest** topics and give them extra time inside the matching phase.
- Create a public GitHub repository named **dotnet-mastery-labs** for small, throw-away experiments (one folder per concept).
- Start a **learning log** (a simple markdown file): date, what you learned, what broke, what you still do not understand.
- Block fixed study slots in your calendar. Consistency beats intensity.

> **Roman Urdu note**
> Pehla hafta plan ko apne hisaab se adjust karne ka hai. Jo topics tumhein pehle se aate hain unpe kam time lagao, aur jahan kamzori hai wahan zyada.

PHASE 1: C# and .NET Internals — Weeks 2-3

Goal: Understand what the runtime is actually doing, so that every later layer (EF Core, ASP.NET Core, performance) stops being magic.

Topic	You should be able to...
async / await and Task	Explain the compiler-generated state machine; use CancellationToken everywhere; avoid sync-over-async (.Result / .Wait()); know when ValueTask helps.
LINQ and collections	Explain deferred execution and multiple-enumeration bugs; know IEnumerable vs IQueryable; choose List / Dictionary / HashSet by complexity.
Generics, delegates, expression trees	Use variance, Func / Action, events; understand how EF Core reads Expression<Func<T,bool>> and turns it into SQL.
Modern C#	Use records, pattern matching, nullable reference types, init / required members, primary constructors, collection expressions.
Memory and GC	Stack vs heap, generations, Large Object Heap, boxing, IDisposable / IAsyncDisposable, Span<T> basics.
Concurrency	Thread pool, locks, SemaphoreSlim, Channel<T>, race conditions and how to reproduce them in a test.

Roman Urdu note

Yahan sab se important async/await hai. Ye clear ho gaya to backend ki aadhi performance aur deadlock problems samajh aa jati hain. Sirf syntax nahi, ye seekho ke andar kya ho raha hai.

Apply it in Mercato

Add CancellationToken through every async path in Mercato. Write a small lab project that reproduces a deferred-execution bug and a sync-over-async deadlock.

Checkpoint (do not move on until you can do this)

Explain, without notes, why .Result can deadlock and what happens step by step when you await a Task.

Resources: C# in Depth (Jon Skeet); Stephen Cleary's blog (async best practices); Pro .NET Memory Management (Konrad Kokosa); Microsoft Learn: C# documentation.

PHASE 2: ASP.NET Core Deep Dive — Weeks 4-6

Goal: Know the framework well enough to predict its behaviour, and to design a clean, consistent HTTP API.

Topic	You should be able to...
Hosting and request pipeline	Explain Kestrel, WebApplication builder, middleware order, custom middleware, short-circuiting.
Dependency Injection	Singleton / Scoped / Transient, captive-dependency bugs, keyed services, scope validation, decorator pattern.
Configuration and Options	appsettings layering, environment variables, user-secrets; IOptions vs IOptionsSnapshot vs IOptionsMonitor; options validation on startup.
Routing and endpoints	Controllers vs Minimal APIs and the trade-offs, route constraints, endpoint filters, endpoint groups.
Model binding and validation	DTO design, FluentValidation, never bind entities directly (over-posting), consistent validation errors.
Error handling	ProblemDetails, IExceptionHandler, one error contract for the whole API, correct status codes.
API design	Pagination / filtering / sorting, versioning, idempotency keys, OpenAPI docs, ETag and conditional requests.
HttpClientFactory and hosted services	Typed clients, BackgroundService, graceful shutdown.

Roman Urdu note
Middleware order ko dil se samjho. CORS, auth aur exception handling ke bohot se bugs sirf order ghalat hone ki wajah se aate hain.

Apply it in Mercato
Mercato Backend Foundation: global exception handling with ProblemDetails, API versioning, Options pattern with validation, a validation pipeline, consistent response contract.

Checkpoint (do not move on until you can do this)
Draw the request pipeline from memory and explain why UseAuthentication must come before UseAuthorization.

Resources: ASP.NET Core in Action (Andrew Lock); Andrew Lock's blog: .NET Escapades; Microsoft Learn: ASP.NET Core fundamentals.

PHASE 3: Data Access Mastery — Weeks 7-10

Goal: Be the person who can look at a slow or incorrect data layer and explain exactly why, using SQL and query plans as evidence.

Topic	You should be able to...
EF Core internals	DbContext lifetime, change tracker, AsNoTracking, identity resolution, how SaveChanges works, how LINQ becomes SQL.
Query performance	Detect and fix N+1, Include vs split queries, projection with Select, compiled queries, reading generated SQL from logs.
Modeling	Fluent configuration, owned types / value objects, TPH vs TPT, value converters, global query filters (soft delete), shadow properties.
Migrations	Team workflow, idempotent SQL scripts, data seeding, safe schema changes with no downtime.
Concurrency and transactions	rowversion tokens, optimistic vs pessimistic locking, isolation levels, deadlocks, transaction pitfalls.
SQL Server skills	Clustered / non-clustered / filtered indexes, execution plans, statistics, keyset vs offset pagination, when a stored procedure is justified.
Dapper and raw SQL	When to bypass EF Core; lightweight read models for the query side of CQRS.
Interceptors and auditing	SaveChanges interceptors, CreatedAt / UpdatedBy, audit log design.

Roman Urdu note
EF Core ko black box mat samjho. Har query ka generated SQL dekhne ki aadat daalo. Yehi aadat average developer aur strong backend developer mein farq banati hai.

Apply it in Mercato
Prevent overselling with variant-level stock and rowversion, concurrency-safe coupon redemption, filtered unique indexes, audit interceptor, keyset pagination on product listing.

Checkpoint (do not move on until you can do this)
Take one slow endpoint, prove the N+1 from logs, fix it, and record query count and response time before and after.

Resources: Microsoft Learn: EF Core performance guidance; Entity Framework Core in Action (Jon P. Smith); use-the-index-luke.com; Brent Ozar's free SQL Server training.

PHASE 4: Authentication and Security — Weeks 11-13

Goal: Treat security as part of design, not a feature added at the end.

Topic	You should be able to...
AuthN fundamentals	Cookies vs bearer tokens, JWT anatomy, signing algorithms, claims, token lifetimes.
Refresh tokens	Rotation, reuse detection, HttpOnly + Secure + SameSite cookies, revocation.
ASP.NET Core Identity	User / role stores, password hashing, lockout, email confirmation, 2FA; when to use it vs a custom implementation.
Authorization	Roles vs policies vs resource-based authorization, requirements and handlers, claims transformation.
OAuth 2.0 and OpenID Connect	Authorization Code + PKCE, external logins (e.g. Google), identity providers such as Keycloak, Entra ID, OpenIddict.
Web security	OWASP Top 10, CORS, CSRF, XSS, SQL injection, SSRF, security headers, rate limiting.
Secrets and data protection	user-secrets, environment variables / Key Vault, Data Protection API, encryption at rest, handling PII.
Payment security	Webhook signature verification, idempotency, reducing PCI scope by using hosted payment pages.

Roman Urdu note

Security baad mein add hone wali cheez nahi, design ka hissa hai. Har feature ke baad sochho: agar attacker ye endpoint direct call kare to kya hoga?

Apply it in Mercato

User / Admin / Super Admin authorization with policies, rotated refresh tokens in HttpOnly cookies, webhook signature verification for the payment provider, rate limiting on auth endpoints.

Checkpoint (do not move on until you can do this)

Attack your own API: try IDOR, token replay and mass assignment. Document each finding and fix it.

Resources: OWASP Top 10 and OWASP Cheat Sheet Series; PortSwigger Web Security Academy (free labs); Microsoft Learn: ASP.NET Core security; Aaron Parecki: OAuth 2.0 Simplified.

PHASE 5: Architecture and Design Patterns — Weeks 14-16

Goal: Choose structure deliberately, and be able to justify every pattern you use.

Topic	You should be able to...
Clean Architecture	Dependency rule, layer responsibilities, avoiding leaky abstractions, enforcing rules with architecture tests.
DDD tactical patterns	Entities, value objects, aggregates and invariants, domain events, ubiquitous language.
CQRS and MediatR	Commands vs queries, pipeline behaviors (validation, logging, transactions), and when NOT to use it.
Result pattern and error modeling	No exceptions for control flow; typed errors mapped to ProblemDetails.
Alternatives	Vertical slice architecture and modular monolith: trade-offs against layered Clean Architecture.
Reliability patterns	Outbox pattern, idempotent handlers, saga / process manager basics, explicit state machines.
Repository and Unit of Work	When DbContext already is one, and when an extra abstraction only adds noise.

Roman Urdu note

Patterns tools hain, goal nahi. Har pattern ke saath poochho: ye kaunsa problem solve kar raha hai, aur is project mein wo problem hai bhi ya nahi?

Apply it in Mercato

Implement the Cart, CheckoutSession, PendingOrder, PaymentAttempt, ConfirmedOrder flow as an explicit state machine with idempotency; MediatR pipeline behaviors; outbox for notifications.

Checkpoint (do not move on until you can do this)

Write three Architecture Decision Records (ADRs) for your project, each comparing your choice against a real alternative.

Resources: Clean Architecture (Robert C. Martin); Domain-Driven Design Distilled (Vaughn Vernon); Jimmy Bogard's blog (vertical slices, MediatR); Milan Jovanovic's blog and courses.

PHASE 6: Performance, Reliability and Real-Time — Weeks 17-19

Goal: Make the system observable, fast where it matters, and resilient when things fail.

Topic	You should be able to...
Logging and observability	Structured logging (Serilog), correlation IDs, OpenTelemetry traces and metrics, dashboards (Seq / Grafana).
Caching	IMemoryCache, Redis distributed cache, HybridCache, output caching, invalidation strategies, cache stampede.
Resilience	Timeouts, retries with jitter, circuit breaker, bulkhead (Polly / Microsoft.Extensions.Resilience).
Background processing	BackgroundService, Hangfire or Quartz, message queues (RabbitMQ), retry and dead-letter handling.
Real-time	SignalR hubs, groups, authentication on hubs, scale-out with a Redis backplane.
Profiling and benchmarking	BenchmarkDotNet, dotnet-counters, dotnet-trace, load testing with k6, reading a flame graph.
Operational basics	Health checks, graceful shutdown, rate limiting, feature flags.

Roman Urdu note
Pehle measure karo, phir optimize. Bina profiler ke optimization sirf andaza hota hai.

Apply it in Mercato
SignalR notifications, Redis caching for the product catalog, Serilog with correlation IDs, load test the checkout flow.

Checkpoint (do not move on until you can do this)
Load test with k6, find the top bottleneck using traces, fix it, and show the improvement in p95 latency.

Resources: Microsoft Learn: .NET observability with OpenTelemetry; Serilog and Polly documentation; BenchmarkDotNet documentation; k6 documentation.

PHASE 7: Testing — Week 20 (and continuous)

Goal: Build a safety net that lets you refactor without fear.

Topic	You should be able to...
Unit testing	xUnit, mocking (NSubstitute / Moq), Arrange-Act-Assert, testing domain logic without mocks.
Integration testing	WebApplicationFactory, Testcontainers with a real SQL Server, test data builders, database reset with Respawn.
Architecture and contract tests	NetArchTest or ArchUnitNET to enforce layer rules; API contract checks.
Test strategy	Testing pyramid vs trophy, what NOT to test, coverage as a guide not a target, mutation testing (Stryker.NET).

Roman Urdu note

Test likhna slow lagta hai, lekin refactor ke waqt yehi tumhari safety net hai.

Apply it in Mercato

Integration tests for the auth and checkout flows; architecture tests that fail if Domain references Infrastructure.

Checkpoint (do not move on until you can do this)

A CI gate that fails the build when a test fails or an architecture rule is broken.

Resources: Unit Testing Principles, Practices, and Patterns (Vladimir Khorikov); Microsoft Learn: integration tests in ASP.NET Core; Testcontainers for .NET documentation.

PHASE 8: DevOps and Deployment — Weeks 21-22

Goal: Ship your own work to a real environment, repeatably.

Topic	You should be able to...
Docker	Multi-stage Dockerfile, docker-compose (API + SQL Server + Redis), image size and security basics.
CI/CD	GitHub Actions build / test / publish, environments and secrets, running migrations safely in a pipeline.
Hosting	Kestrel behind Nginx or YARP, forwarded headers, HTTPS, Azure App Service or a VPS deployment.
Environments and configuration	Dev / staging / prod parity, twelve-factor principles, secrets management.
Monitoring and rollback	Health probes, alerts, rolling and blue-green deployment concepts, rollback plans.

Roman Urdu note
Code sirf tumhare laptop pe chale to wo finished nahi hai. Ek baar poora pipeline khud bana lo, phir deployment ka dar khatam ho jata hai.

Apply it in Mercato
Containerize Mercato, set up a GitHub Actions pipeline, deploy to a staging environment.

Checkpoint (do not move on until you can do this)
A push to main automatically runs tests, builds the image and deploys to staging.

Resources: Microsoft Learn: Docker for .NET; GitHub Actions documentation; The Twelve-Factor App (12factor.net).

PHASE 9: Advanced and Distributed Systems — Weeks 23-24

Goal: Know the concepts well enough to make (and refuse) architectural decisions with confidence.

Topic	You should be able to...
Distributed systems basics	CAP, eventual consistency, idempotency, why distributed transactions are avoided.
Microservices vs modular monolith	Boundaries, when to split, API gateway (YARP), service-to-service authentication.
Messaging and events	RabbitMQ / Kafka concepts, integration events, sagas, reliable delivery with the outbox.
gRPC and alternatives	gRPC, GraphQL (Hot Chocolate), and when plain REST is enough.
Multi-tenancy and scaling	Tenant isolation strategies, stateless services, horizontal scaling, read replicas.
Cloud-native overview	Kubernetes basics, .NET Aspire for local orchestration.

Roman Urdu note
Microservices ki jaldi mat karo. Pehle modular monolith achhe se banana seekho. Zyada tar projects ko microservices ki zaroorat hoti hi nahi.

Apply it in Mercato
Extract the notification module behind an event bus as an experiment; try .NET Aspire for local development.

Checkpoint (do not move on until you can do this)
Write a design document describing how Mercato would scale to 10x traffic, including what you would NOT change.

Resources: Designing Data-Intensive Applications (Martin Kleppmann); Microservices Patterns (Chris Richardson); microservices.io.

How to Study Each Week

Block	Share	What to do
Learn	25%	Read docs or a book chapter, watch one focused talk. Take notes in your own words, not copied text.
Build	50%	Implement the concept in Mercato or a lab project. Ship something small every week.
Break	10%	Deliberately cause the failure: race condition, N+1, expired token, wrong middleware order. See the symptom yourself.
Explain	15%	Write a short note or explain aloud as if teaching a junior. If you cannot explain it, you have not learned it yet.

Rules That Keep the Plan Honest

- **Avoid tutorial hell.** After any tutorial, rebuild the same thing from a blank project without looking.
- **Read the source of truth.** Official Microsoft docs and the framework source on GitHub come before blog posts.
- **Measure before you claim.** Every performance or security claim needs a number, a log or a test behind it.
- **Do not over-engineer.** If you cannot name the problem a pattern solves in your project, leave the pattern out.
- **Do a short weekly review.** Five minutes on Sunday: what moved forward, what is blocked, what is next.
- **Keep the plan alive.** If a phase runs long, extend it; do not skip its checkpoint.

> **Roman Urdu note**
> Roz thora sa kaam, hafte mein ek chhota deliverable. 24 hafte baad tumhare paas notes, repo aur ek deployed project hoga, sirf ye ehsas nahi ke 'mujhe sab aata hai'.

Progress Tracker

Phase	Planned	Started	Checkpoint passed	Notes
0. Baseline audit	Week 1			
1. C# and .NET Internals	Weeks 2-3			
2. ASP.NET Core Deep Dive	Weeks 4-6			
3. Data Access Mastery	Weeks 7-10			
4. Authentication and Security	Weeks 11-13			
5. Architecture and Design Patterns	Weeks 14-16			
6. Performance, Reliability and Real-Time	Weeks 17-19			
7. Testing	Week 20 (and continuous)			
8. DevOps and Deployment	Weeks 21-22			
9. Advanced and Distributed Systems	Weeks 23-24			

Self-Assessment Checklist

Rate each topic from 1 to 5 now (Week 1), again at Week 12, and again at Week 24. 1 = never used, 3 = can use with docs, 5 = can teach it and explain the trade-offs.

Phase	Topic	Week 1	Week 12	Week 24
P1	async / await and Task			
P1	LINQ and collections			
P1	Generics, delegates, expression trees			
P1	Modern C#			
P1	Memory and GC			
P1	Concurrency			
P2	Hosting and request pipeline			
P2	Dependency Injection			
P2	Configuration and Options			
P2	Routing and endpoints			
P2	Model binding and validation			
P2	Error handling			
P2	API design			
P2	HttpClientFactory and hosted services			
P3	EF Core internals			
P3	Query performance			
P3	Modeling			
P3	Migrations			
P3	Concurrency and transactions			
P3	SQL Server skills			
P3	Dapper and raw SQL			
P3	Interceptors and auditing			
P4	AuthN fundamentals			
P4	Refresh tokens			
P4	ASP.NET Core Identity			
P4	Authorization			
P4	OAuth 2.0 and OpenID Connect			
P4	Web security			
P4	Secrets and data protection			

Phase	Topic	Week 1	Week 12	Week 24
P4	Payment security			
P5	Clean Architecture			
P5	DDD tactical patterns			
P5	CQRS and MediatR			
P5	Result pattern and error modeling			
P5	Alternatives			
P5	Reliability patterns			
P5	Repository and Unit of Work			
P6	Logging and observability			
P6	Caching			
P6	Resilience			
P6	Background processing			
P6	Real-time			
P6	Profiling and benchmarking			
P6	Operational basics			
P7	Unit testing			
P7	Integration testing			
P7	Architecture and contract tests			
P7	Test strategy			
P8	Docker			
P8	CI/CD			
P8	Hosting			
P8	Environments and configuration			
P8	Monitoring and rollback			
P9	Distributed systems basics			
P9	Microservices vs modular monolith			
P9	Messaging and events			
P9	gRPC and alternatives			
P9	Multi-tenancy and scaling			
P9	Cloud-native overview			